

多态

1. 通用概念:同一论域中一个元素可有多种解释。
 2. 提高面向对象设计的语言灵活性
 3. 程序设计语言:OO程序设计
 4. 多态形式
 1. 函数重载:(静态多态), 和虚函数的动态多态不同(一名多用):函数重载包含操作符重载
 2. 类属多态:模板:template
- [1. 操作符重载](#)
 - [1.1. 操作符 + 的重载](#)
 - [1.2. 可以重载的操作符](#)
 - [1.3. 双目操作符的重载](#)
 - [1.3.1. 类成员函数\(双目操作符\)](#)
 - [1.3.2. 全局函数](#)
 - [1.4. 返回值总结](#)
 - [1.5. 单目操作符的重载](#)
 - [1.6. 操作符 = 的重载](#)
 - [1.7. 操作符 \[\] 的重载](#)
 - [1.8. 多维数组 class Array2D](#)
 - [1.8.1. 多维数组的最终版本](#)
 - [1.9. 操作符 \(\) 的重载](#)
 - [1.9.1. 函数调用](#)
 - [1.9.2. 操作符类型转换的重载](#)
 - [1.10. 操作符 -> 的重载](#)
 - [1.11. 操作符 new 和 delete 的重载](#)
 - [1.11.1. 重载 new](#)
 - [1.11.2. 重载 delete](#)
 - [1.11.3. new和delete考试](#)
 - [2. 模板 template](#)
 - [2.1. 模板](#)
 - [2.2. 类属函数 templat function](#)
 - [2.3. 函数重载](#)
 - [2.4. 函数指针](#)
 - [2.5. 函数模板](#)
 - [2.6. 类模板](#)
 - [2.7. 模板例子](#)
 - [2.8. C++中模板的完整定义通常出现在头文件中](#)
 - [2.9. Template MetaProgramming 元编程](#)

- [3. 参考](#)

1. 操作符重载

1. 函数重载

1. **名同、参数不同**，返回值不同没有用的:参数顺序、参数类型匹配(找到最佳匹配)
2. 静态绑定

2. 歧义控制:

1. 顺序:
2. 最佳匹配:
 1. 原则一:这个匹配每一个参数不必其他的匹配更差
 2. 原则二:这个匹配有一个参数更精确匹配
3. 整形提升:更好的，标准转换(标准转换都是一视同仁的)
4. 窄转换?允许的，大->小

3. 操作符重载(变为一种函数)

1. 动机:操作符语义
 - built_in 类型
 - 自定义数据类型
2. 作用:
 1. 提高可读性
 2. 提供可扩充性
4. 重点记忆返回的变量

1.1. 操作符 + 的重载

1. 重载第一步

```
1 class Complex {
2     double real, imag;
3     public:
4         Complex() {real = 0; imag = 0;}
5         Complex(double r, double i) { real = r; imag = i; }
6         Complex add(Complex& x);
7 };
8 Complex a(1,2),b(3,4),c;
9 c=a.add(b);//想要写成 a + b
10 //使用操作符重载
11 class Complex {
12     double real, imag;
13     public:
14         Complex() { real = 0; imag = 0; }
15         Complex(double r, double i) { real = r; imag = i; }
16         Complex operator + (Complex& x) {
17             Complex temp;
18             temp.real = real + x.real;
19             temp.imag = imag + x.imag;
20             return temp;
```

```

21     }
22 };
23 Complex a(1,2),b(3,4),c;
24 c = a.operator + (b);
25
26 //进一步完成操作符重载
27 class Complex {
28     double real, imag ;
29     public :
30         Complex() { real = 0 ; imag = 0 ; }
31         Complex(double r, double i) {
32             real = r;
33             imag = i;
34         }
35         friend Complex operator+(Complex& c1 , Complex& c2); //这个是已经预定义
        好的，我们这样子写就是重载
36 };
37 //全局函数
38 Complex operator+ (Complex& c1 , Complex& c2 ) { //全局函数重载至少包含一个用户自定义类型
39     Complex temp;
40     temp.real = c1.real + c2.real;
41     temp.imag = c1.imag + c2.imag;
42     return temp;
43 } //一般返回临时变量
44
45 Complex a(1,2),b(3,4),c;
46 c = a + b; //自动进行翻译

```

2. 自增和自减的问题

```

1 class Counter {
2     int value;
3     public:
4         Counter() { value = 0; }
5         Counter& operator ++() //++a 左值
6         {
7             value ++;
8             return *this;
9         }
10        Counter operator ++(int) //a++ 右值
11        {
12            Counter temp = *this;
13            value++;
14            return temp;
15        }
16 }

```

3. 重载++函数:封装SAT的问题

1. 返回值引用或者是值是有区别的

```

1 enum Day { SUN, MON, TUE, WED, THU, FRI, SAT};
2 Day& operator++(Day& d)
3 { return d= (d==SAT)? SUN: Day(d+1); }

```

```

4 //重载重定向符号, 用的很多, 不能进成员函数重载
5 ostream& operator << (ostream& o, Day& d)
6 {   switch (d)
7     {   case SUN: o << "SUN" << endl; break; //直接使用ostream中的<<
8         case MON: o << "MON" << endl; break;
9         case TUE: o << "TUE" << endl; break;
10        case WED: o << "WED" << endl; break;
11        case THU: o << "THU" << endl; break;
12        case FRI: o << "FRI" << endl; break;
13        case SAT: o << "SAT" << endl; break;
14    }
15    return o; //为什么要return ostream类型的变量: 需要连续的使用可以链式调用, Cout <<
16    1 << 2;
17 }
18 void main(){
19     Day d = SAT;
20     ++d;
21     cout << d;
22 }

```

1.2. 可以重载的操作符

1. 不可以重载的操作符: `.` (成员访问操作符)、`.*` (成员指针访问运算符, 如下)、`::` (域操作符)、`?:` (条件操作符)、`sizeof`: 也不重载

1. 原因: 前两个为了防止类访问出现混乱
2. `::` 后面是名称不是变量
3. `?:` 条件运算符涉及到跳转, 如果重载就影响了理解

```

1 class A
2 {   int x;
3     public:
4         A(int i): x(i) {}
5         void f() {}
6         void g() {}
7 };
8 void (A::*p_f)(); //A类成员的函数指针
9
10 p_f = &A::f;
11 (a.*p_f)();
12 int a = 0; b = 0;
13 b?(a = 1):(b = 1); //a == b == 1
14 operator ?: (p, a = 1, b = 1) //均执行了

```

1. 重载基本原则:

1. 方法: (大多数都支持, 但是有的不支持)
 1. 类成员函数
 2. 带有类参数的全局函数
2. 遵循原有语法
 1. 单目/双目: 一一对应
 2. 优先级

3. 结合性

2. 永远不要重载 `&&` 和 `||`: 会造成极大的问题

1.3. 双目操作符的重载

1.3.1. 类成员函数(双目操作符)

1. 类成员函数:

1. 格式: `<ret type> operator #(<arg>)`

2. `this`: 隐含, 必然是第一个参数

2. 使用:

```
1 <class name> a,b;  
2 a # b; // a -> this  
3 a.operator#(b)
```

1.3.2. 全局函数

1. 友元: `friend <ret type> operator #(<arg1>, <arg2>)`

2. 格式: `<ret type> operator #(<arg1>, <arg2>)`

3. 注意: `=`、`()`、`[]`、`->` 不可以作为全局函数重载

- 大体上来讲, C++ 一个类本身对这几个运算符就已经有了相应的解释了。
- 如果将这四种符号进行友元全局重载, 则会出现一些冲突
- 下标和箭头运算符为什么? 有保留调用顺序, 我们希望能保留原来的顺序, 而全局不能要求, 而成员函数的 `this` 就可以解决这个问题
- [参考](#)

4. 全局函数作为补充:

1. 单目运算符最好重载为类的成员函数
2. 双目运算符最好重载为类的友元函数

```
1 class CL {  
2     int count;  
3     CL(int i){...} // 10 可以直接隐式类型转换  
4     public:  
5         friend CL operator +(int i, CL& a);  
6         friend CL operator +(CL& a, int i);  
7 }; // 支持隐式类型转换就行  
8 // 如果最左边不是类对象, 则必须作为友元函数
```

5. 永远不要重载 `&&` 和 `||`: 逻辑与和逻辑或

1. 原因: 短路, 类似?:
2. 虽然绝大多数都没有问题, 但是如果有逻辑短路容易出现问题

```

1 char *p;
2 if ((p != 0) && (strlen(p) >10)) //利用了短路，一旦计算就没有短路行为了
3 if (expression1 && expression2)
4 if (expression1.operator && (expression2))
5 if (operator && (expression1, expression2))

```

1. 返回类型的问题:如果没有&的时候，第一个return出现了对象拷贝，避免:临时变量不能返回拷贝

```

1 class Rational {
2     public:
3         Rational(int,int);
4         const Rational& operator *(const Rational& r) const; //const写不写
        都行，写了更好
5     private:
6         int n, d;
7 };
8 // operator * 的函数体
9 return Rational(n * r.n, d * r.d);
10
11 Rational *result = new Rational(n*r.n, d*r.d);
12 return *result; //返回引用的问题?
13 // w = x * y * z 出现问题:出现内存泄露的问题
14
15 static Rational result; //声明为static
16 result.n = n * r.n;
17 result.d = d * r.d;
18 return result; //static是全局的，可以吗?不可以，同时出现两个的结果会出现问题
19 //if((a * b) == (c * d)) ->永真式

```

2. 操作符重载的哲理:尽量让事情有效率，但不是过度有效率(返回引用)
3. 结论:每次就是返回一个拷贝，而不是引用

1.4. 返回值总结

1. 加减乘除:就是拷贝，不是引用，效率不太高?为了解决这个问题:可以返回值优化，第一个return没有拷贝，直接返回的是一个对象(无拷贝)，先计算，最后生成一个对象返回。

1.5. 单目操作符的重载

1. 类成员函数:
 1. this:隐含
 2. 格式: <ret type> operator#() :this的隐含
2. 全局函数:
 1. <ret type> operator#(<arg>)
 2. 参数必须为自定义类型
3. 单目操作符在绝大多数情况下重载为类的成员函数
4. 15min没了
5. a++ 和 ++ a

```

1 class Counter{

```

```

2      int value;
3      public:
4          Counter() { value = 0; }
5          Counter& operator ++() // ++a
6          {
7              value++;
8              return *this;
9          }
10         Counter operator ++(int) //a++
11         {
12             Counter temp=*this;//这里的int值是什么意思?区分两个函数, dummy
13             //argument, 哑元变量
14             value++;
15             return temp;
16         }
17     }

```

1.6. 操作符 = 的重载

1. 默认赋值操作符重载函数
2. 逐个成员赋值
3. 对含有对象成员的类, 该定义是递归的
4. 赋值操作符的重载不可以被继承: 因为拷贝构造, 派生出来的类有一些新的部分
5. 返回引用类型: 返回*this的引用, 支持链式赋值
6. this引用应该是非常量引用, 返回出来的是作为右值进行计算
 1. a = b = c: 不要求非常量引用
 2. (a = b).f(): 要求非常量引用
7. 例一:

```

1  class A;
2  A a = b; //需要调用拷贝构造函数(更重要的是构造, 在构造对象时候调用)
3  A a(b);
4  A a,b;
5  a = b; //需要调用

```

```

1  class A {
2      int x,y ;
3      char *p ;
4      public :
5          A(int i,int j,char *s):x(i),y(j){
6              p = new char[strlen(s)+ 1 ];
7              strcpy(p,s); //进行拷贝, 最后留一个\0
8          }
9          virtual ~A(){
10             delete[] p;
11         }
12         A& operator = (A& a) {
13             //赋值
14             x = a.x;
15             y = a.y;
16             delete []p;
17             p = new char[strlen(a.p)+1];

```

```

18         strcpy(p,a.p);
19         return *this;//也会出现悬垂
20     }//还有问题，就是赋值自身会出现问题
21 };
22 A a, b;
23 a = b;//调用自己的复制
24
25 //idle pointer, B被析构的时候会将p释放掉，导致p指向已经被释放掉的指针
26 //Memory leak,A申请的区域可能没有办法被释放

```

```

1 //更安全的拷贝，先new再delete
2 char *porig = p;
3 p = new char ...
4 strcpy();
5 delete porig;
6 return *this;
7 //自我赋值可以吗？可以，换了一块内存空间，没有内存泄露

```

1. 注意:避免自我赋值(因为是相同的内存地址)

1. Sample: class string

2. s = s

1. class {... A void f(A& a);...}

2. void f (A& a1, A& a2)

3. int f2(Derived &rd,Base& rb);

3. Object identity

1. Content

2. Same memory location

3. Object identifier

```

1 if(this == &a)
2     return *this;
3 delete p;//48min - 50min

```

```

1 class A{
2     public:
3         ObjectID identity() const;
4 };
5 A *p1,*p2;
6 p1-> identity() == p2-> identity()

```

1.7. 操作符 [] 的重载

```

1 class string {
2     char *p;
3     public :
4         string(char *p1){
5             p = new char [strlen(p1)+ 1];
6             strcpy(p,p1);//#pragma warning(disable:4996)来屏蔽问题

```



```

7         }
8         char& operator[](int i){
9             return p[i];
10        }
11        const char operator[](int i) const{
12            return p[i];
13        }
14        //可以用两个重载函数吗?是可以的
15        virtual ~string() { delete[] p ; }
16    };
17    string s("aacd");
18    s[2] = 'b' ;
19    //第一个重载加上const可以使得const或者非const对象都可以调用
20    const string cs('const');
21    cout << cs[0];
22    const cs[0] = 'd';//const 版本不想被赋值(返回const的), 非const版本想要被赋值, 之后再
    进行重载的时候就需要同时重载两个

```

1.8. 多维数组 class Array2D



```

1  class Array2D{
2      int n1, n2;
3      int *p;
4      public:
5          Array2D(int l, int c):n1(l),n2(c){
6              p = new int[n1*n2];
7          }
8          virtual ~Array2D() { delete[] p; }
9      };
10
11  int & Array2D::getElem(int i, int j) { ... }
12  //上面是实现多维数组
13  Array2D data(2,3);
14  data.getElem(1,2) = 0;
15  //target -> data[1][2]
16  //想法:化解为两次调用
17  data.operator[](1)[2];//int *operator[](int i) 一次偏移一行, 转化成Array1D
18  data.operator[](1).operator[](2)
19
20  //问题:三维数组重载问题:重载一次降维一次, 3D->2D等等, 多个依次进行重载, 重载之后返回对象
21  //代理对象:Array1D
22
23  class Array1D{
24      int *q;//一维降低到int *就行
25      Array1D(int *p){ q = p; }
26      int& operator[](j){
27          return q[j];
28      }
29  }

```

1.8.1. 多维数组的最终版本

```
1  class Array2D{
2      private:
3          int *p;
4          int num1, num2;
5      public:
6          class Array1D{//Surrogate 多维, proxy class
7              public:
8                  Array1D(int *p) { this->p = p; }
9                  int& operator[ ] (int index) { return p[index]; }
10                 const int operator[ ] (int index) const { return p[index]; }
11             private:
12                 int *p;
13             };
14             Array2D(int n1, int n2) {
15                 p = new int[n1 * n2];
16                 num1 = n1;
17                 num2 = n2;
18             }
19             virtual ~Array2D() {
20                 delete [] p;
21             }
22             Array1D operator[](int index) {
23                 return p + index * num2;//return的值和int*相同, 构造函数不能声明成显式
构造函数。
24             }
25             //这里为什么是array1d?通过构造函数进行类型转换
26             const Array1D operator[ ] (int index) const {
27                 return p+index*num2;
28             }
29     };
```

1.9. 操作符 () 的重载

1. ()的意义

1. 函数调用
2. 类型转换操作符

1.9.1. 函数调用

```
1  class Func {
2      double para;
3      int lowerBound , upperBound ;
4      public:
5          double operator()(double,int,int);
6  };
7  Func f;
8  f(2.4, 0, 8);
9  class Array2D{
10     int n1, n2;
11     int *p;
12     public:
```

```

13     Array2D(int l, int c):n1(l),n2(c){
14         p = new int[n1*n2];
15     }
16     virtual ~Array2D() {
17         delete[] p;
18     }
19     int& operator()(int i, int j){
20         return (p+i*n2)[j]; //优化getElement
21     }
22 };

```

1.9.2. 操作符类型转换的重载

1. 基本数据类型
2. 自定义类

```

1  class Rational {
2      public: Rational(int n1, int n2) {
3          n = n1;
4          d = n2;
5      }
6      operator double() { //类型转换操作符, 语法特殊
7          return (double)n/d;
8      }
9      private:
10         int n, d;
11 };
12 //减少混合计算中需要定义的操作符重载函数的数量
13 Rational r(1,2);
14 double x = r;
15 x = x + r; //避免的double 和 rational 的全局函数重载, 会自动全部转换为double

```

```

1 //48min
2 ostream f("abc.txt");
3 if (f)
4 //重载 数值型: 如 int

```

1. 问题:为什么禁止在类外禁止重载赋值操作符?
 1. 如果没有类内提供一个赋值操作符, 则编译器会默认提供一个类内的复制操作符
 2. 查找操作符优先查找类内, 之后查找全局, 所以全局重载赋值操作符不可能被用到

1.10. 操作符 -> 的重载

1. ->为二元运算符, 重载的时候按照一元操作符重载描述。

```

1 A a;
2 a->f();
3 a.operator->(f)
4 a.operator->()->f() //重载时按一元操作符重载描述, 这时, a.operator->() 返回一个指针 (或者是已经重定义过->的对象)

```

2. 例子:画图板程序

```

1  class CPen {
2      int m_color;
3      int m_width;
4      public:
5          void setColor(int c){ m_color = c;}
6          int getWidth(){ return m_width; }
7  };
8  class CPanel {
9      CPen m_pen;
10     int m_bkColor;
11     public:
12         CPen* getPen(){return &m_pen;}
13         void setBkColor(int c){ m_bkColor =c;}
14 };
15 CPanel c;
16 c.getPen()->setColor(16);
17 //简单修改，可以返回一个对象内部对象的指针
18 class CPen {
19     int m_color;
20     int m_width;
21     public:
22         void setColor(int c){ m_color = c;}
23         int getWidth(){return m_width; }
24 };
25 class CPanel {
26     CPen m_pen;
27     int m_bkColor;
28     public:
29         CPen* getPen(){return &m_pen;}
30         void setBkColor(int c) { m_bkColor =c;}
31 };
32 CPanel c;
33 c->setColor(16);
34 //等价于
35 //c.operator->()->setColor(16);
36 //c.m_pen.setColor(16)
37 c->getWidth();
38 //等价于
39 //c.operator->()->getWidth();
40 //c.m_pen.getWidth()
41 CPanel *p=&c;
42 p->setBkColor(10);

```

3. Prevent memory Leak:需要符合compiler控制的生命周期

```

1  class A{
2      public:
3          void f();
4          int g(double);
5          void h(char);
6  };
7  void test(){
8      A *p = new A;
9      p->f(); //如果出错可能会导致问题
10     p->g(1.1); //返回值

```

```

11     p->h('A');
12     delete p;
13 }
14 //更好的管理A对象，不用在任何退出的地方写delete p
15 void test()
16 {
17     Awrapper p(new A);
18     p->f(); //如果出错可能会导致问题
19     p->g(1.1); //返回值
20     p->h('A');
21     delete p;
22 }
23 //须符合compiler控制的生命周期
24 class Awrapper{ //不包含逻辑
25     A* p; // ? T p; 支持多个类型
26     public:
27         Awrapper(A *p) { this->p = p; }
28         ~Awrapper() { delete p; }
29         A* operator->() { return p; }
30 }; //RAII 资源获取及初始化
31 //函数返回，销毁局部指针的时候会直接删除

```

1.11. 操作符 new 和 delete 的重载

1. 频繁调用系统的存储管理，影响效率。
2. 程序自身管理内存，提高效率
3. 方法:
 1. 调用系统存储分配，申请一块较大的内存
 2. 针对该内存，自己管理存储分配、去配
 3. 通过重载new与delete来实现
 4. 重载的new与delete是静态成员(隐式的，不需要额外声明，不允许操作任何类的数据成员)
 5. 重载的new与delete遵循类的访问控制，可继承(注意派生类和继承类的大小问题，开始5min左右)
4. 我们想要对某些程序进行自己的资源管理的话，可以重载这两个操作符。
5. 有些我们重复新建销毁的，比如Restful的可以单独管理
6. new构造和返回指针
7. delete析构和释放内存
8. 可以重载成全局函数，也可以重载成类成员函数

1.11.1. 重载 new

1. `void *operator new (size_t size, s...)`
2. 名: operator new
3. 返回类型: void *
4. 第一个参数: size_t(unsigned int)
 - 系统自动计算对象的大小，并传值给size
5. 其他参数: 可有可无

- `A *p = new (...) A`, 表示传给new的

6. new的重载可以有多种

7. 如果重载一个new, 那么通过new动态创建该类的对象时将不再调用内置的(预定义的)new

8. 允许进行全局重载, 但是不推荐使用全局重载

```
1  if(size != sizeof(base))
2      return ::operator new (size); //调用全局标准库的new进行size的分配, 标准库的new永
    远是可以使用的
3  operator new;
4  new A[10];
5  operator new [];
6  void * operator new (size_t size, void*) //是不可以被重载的, 标准库版本
7  void * operator new (size_t size, ostream & log); //可以同时写入到日志
8  void * operator new (size_t size, void * pointer); //定位new, placement new, 被调
    用的时候, 在指针给定的地方的进行new(可能预先已经分配好的), 分配比较快, 长时间运行不被打断(不
    会导致内存不足)
```

8. new也可以new在栈上

```
1  class A{};
2  char buf[sizeof(A)];
3  A* a = new(buf) A; //定位new, 不用分配内存, 直接使用buf指向的区域
```

1.11.2. 重载 delete

1. `void operator delete(void *, size_t size)`

2. 名: operator delete

3. 返回类型: void

4. 第一个参数: void *(必须): 被撤销对象的地址

5. 第二个参数: 可有可无; 如果有, 则必须为size_t类型: 被撤销对象的大小

6. delete 的重载只能有一个

7. 如果重载了delete, 那么通过 delete 撤消对象时将不再调用内置的(预定义的)delete

8. 动态删除其父类的所有的。

9. 如果子类中有一个虚继承函数, 则size_t大小会根据继承情况进行确定大小

1.11.3. new和delete考试

1. 主要考核集中在这些上面

2. 模板 template

2.1. 模板

1. 模板是一种**源代码复用**机制

2. 参数化模块:

- 对程序模块(如:类、函数)加上**类型参数**
- 对不同类型的数据实施相同的操作

3. 实例化:生成具体的函数/类
4. 模板定义了若干个类，需要显式实例化
5. 编译系统自动实例化函数模板：是否实例化模板的某个实例由使用点来决定；如果未使用到一个模板的某个实例，则编译系统不会生成相应实例的代码。

2.2. 类属函数 `templat function`

1. 同一函数对不同类型的数据完成相同的操作
2. 宏实现:

1. `#define max(a,b) ((a)>(b)?(a):(b))`

2. 缺陷:

1. 只能实现简单的功能
2. 没有类型检查

2.3. 函数重载

```
1 int max(int,int);
2 double max(double,double);
3 A max(A,A) ;
```

1. 缺陷:

1. 需要定义的重载函数太多
 2. 定义不全
2. 不可以只有返回值不同

2.4. 函数指针

```
1 void sort(void * , unsigned int, unsigned int, int (* cmp) (void *, void *) )
```

1. 缺陷:

1. 需要定义额外参数
 2. 大量指针运算
 3. 实现起来复杂
 4. 可读性差
2. template更加结构清晰，实现简单

2.5. 函数模板

```
1 //int和double都可以使用，编译器编译的并不是之下的代码，而是T转化成具体代码，然后分别编译
2 template <typename T>
3 void sort(int A[], unsigned int num) {
4     for(int i=1; i<num; i++)
5         for (int j=0; j< num - i; j++) {
6             if (A[j] > A[j+1]) {
7                 T t = A[j];
8                 A[j] = A[j+1];
```

```

9         A[j+1] = t;
10     }
11 }
12 }
13 class C {...}
14 C a[300];
15 sort(a, 300); //没有重载>

```

1. 必须重载操作符 >
2. 函数模板定义了一类重载的函数
3. 函数模板的实例化:
 1. 隐式实现
 2. 根据具体模板函数调用
4. 函数模板的参数
 1. 可多个类型参数，用逗号分隔
 2. 可带普通参数
 - 必须列在类型参数之后
 - 调用时需显式实例化，使用默认参数值可以不显式实例化

```

1 template <class T1, class T2>
2 void f(T1 a, T2 b) {}
3 template <class T, int size>
4 void f(T a) {T temp[size];}
5 f<int,10>(1);

```

1. 函数模板与函数重载配合使用(编译器优先使用没有使用模板的函数)

```

1 template <class T> T max(T a, T b) {
2     return a>b?a:b;
3 }
4 int x, y, z;
5 double l, m, n;
6 z = max(x,y);
7 l = max(m,n);
8 //为了解决max(x,m)我们使用函数重载更新
9 double max(int a, double b) {
10     return a>b? a : b;
11 }

```

2.6. 类模板

1. 类定义带有类型参数，类属类需要显式实例化
2. 类模板中的静态成员属于实例化后的类
3. 类模板的实例化:创建对象时显式指定

```

1 class Stack {
2     int buffer[100];
3     public:
4         void push(int x);

```



```

5         int pop();
6     };
7     void Stack::push(int x) {...}
8     int Stack::pop(){...}
9
10    Stack st1;
11
12    template <class T>
13    class Stack {
14        T buffer[100];
15    public:
16        void push( T x);
17        T pop();
18    };
19
20    template <class T>
21    void Stack <T> ::push(T x) {...}
22
23    template <class T>
24    T Stack <T> ::pop() {...}
25
26    //如下是显式实例化
27    Stack <int> st1;
28    Stack <double> st2;

```

2.7. 模板例子

```

1     template <class T, int size>
2     class Stack {
3         T buffer[size];
4     public:
5         void push(T x);
6         T pop();
7     };
8
9     template <class T, int size>
10    void Stack <T, size>::push(T x) {...}
11
12    template <class T, int size>
13    T Stack <T, size>::pop() {...}
14
15    Stack <int, 100 > st1 ;//上面改为template<class T = int,int size = 100>,这里可以改成stack<> st1用来显示实例化
16    Stack <double, 200 > st2 ;

```

1. 类型参数也可以给出初始值，模板类如果不按照从右往左指定默认值参数，会导致编译错误
2. 函数模板的默认值不一定是从右向左的，C++11之后函数模板才接受默认值参数。
3. 总而言之从右向左给出默认值总是没有问题的。

2.8. C++中模板的完整定义通常出现在头文件中

1. 如果在模块A中要使用模块B中定义的某模板的实例，而在B中未使用这个实例，则模板无法使用这个实例

2. 为什么C++中模板的完整定义常常出现在头文件中?

```
1 //file1.h
2 template <class T> class S {
3     T a;
4     public:
5         void f();
6 };
7
8 //file1.cpp
9 #include "file1.h"
10 template <class T>
11 void S<T>::f(){...}
12
13 template <class T>
14 T max(T x, T y){return x>y?x:y;}
15 void main() {
16     int a,b;
17     max(a,b); //实例化函数模板
18     S<int> x;
19     x.f();
20 }
21
22 //file2.cpp
23 #include "file1.h"
24 extern double max(double,double);
25 void sub(){
26     max(1.1,2.2); //error
27     S<float> x;
28     x.f(); //error
29 }
30 //不能通过编译, 为什么? file2.cpp找不到max定义, 也找不到完整的S代码
```

- 解决方案:将file1.cpp中的代码放置到头文件中
- 连接器可以去掉多重定义

2.9. Template MetaProgramming 元编程

1. 元程序就是编写一些直接可以生成或者操作其他程序的程序, 要在更高层次上。
2. 编写元程序就是元编程(两级编程), 在编译的时候就已经完成编程

```
1 template<int N>
2 class Fib
3 {
4     public:
5         enum { value = Fib<N - 1>::value + Fib<N - 2>::value };
6 };
7
8 //模板显式实例化
9 template<>
10 class Fib<0>{
11     public:
12         enum { value = 1 };
13 };
```

```
14 template<>
15 class Fib<1>
16 {
17     public:
18         enum { value = 1 };
19 };
20 void main() {
21     cout << Fib<8>::value << endl; // calculated at compile time
22 }
```

1. 元编程的特点

1. 输入就是模板中的参数(int N)
2. 返回值往往是enum、static、final等等
3. 往往是只支持整数，但是浮点数也是可以的

2. 选择和循环语句如何操作?

1. 选择可以通过特殊实例化实现: `class isTen<N==10>` :模板的特例化
2. 递归的调用模板就提供了循环的能力

3. 模板元编程是图灵完备的

4. 不作为考核内容

3. 参考

1. [C++泛型与多态\(1\): 基础篇](#)